

UNIT - 4

Control Structures

INTRODUCTION

Generally, C program contains many statements that are normally executed in sequence or we can say in a specific order as per written in a program.

That's why C is also called a sequential language or procedural language because the execution flow is always in a sequence.

INTRODUCTION

In some situation we required to change the execution flow of control according to certain condition. Based on that conditions we want to execute the statements of the program.

- (1) Decision making statement or Conditional Statement (If ... else statement)
- (2) Different branches statement (Switch Case)
- (3) Jumping statement (goto statement)

INTRODUCTION

- **Different ways to write conditional statement (If...else)**

(1) Simple if statement (Without else statement)

(2) if...else statement

(3) else if ladder

(4) Nested if...else statement

1. SIMPLE IF STATEMENT

If statement is the simplest form of decision control statements that is frequently used in decision making. The general form of a simple if statement is shown in the figure.

SYNTAX OF IF STATEMENT

```
if (test expression)
{
    statement 1;
    .....
    statement n;
}
statement x;
```

1. SIMPLE IF STATEMENT

First the test expression is evaluated. If the test expression is true, the statement of if block (statement 1 to n) are executed otherwise these statements will be skipped and the execution will jump to statement x.

1. SIMPLE IF STATEMENT

In some cases, we want to execute the statement only if the condition becomes true. If condition is wrong then we just want to move on next statement.

In such a situation we don't require else statement we can write a program using only if statement with its block.

1. SIMPLE IF STATEMENT EXAMPLE

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a;
    clrscr();
    printf("\n Enter Value : ");
    scanf("%d",&a);
    if(a>5)
    {
        printf("\n Value is Greater than Five");
    }
}
```

2. IF ELSE STATEMENT

The if else is actually just an expansion of the general format of simple if statement.

SYNTAX OF IF ELSE STATEMENT

```
if (test expression)
{
    statement_block 1;
}
else
{
    statement_block 2;
}
statement x;
```

2. IF ELSE STATEMENT

In the if-else construct, first the test expression is evaluated. If the expression is true, statement block 1 is executed and statement block 2 is skipped.

Otherwise, if the expression is false, statement block 2 is executed and statement block 1 is ignored.

In any case after the statement block 1 or 2 gets executed the control will pass to statement x. Therefore, statement x is executed in every case.

2. IF ELSE STATEMENT EXAMPLE

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a,b;
    clrscr();
    printf("\n Enter Value 1 : ");
    scanf("%d",&a);
    printf("\n Enter Value 2: ");
    scanf("%d",&b);
    if(a>b)
    {
        printf("\n A is Maximum");
    }
    else
    {
        printf("\n B is Maximum");
    }
}
```

3. IF ELSE IF STATEMENT

C language supports if else if statements to test additional conditions apart from the initial test expression. The if-else-if construct works in the same way as a normal if statement.

3. IF ELSE IF STATEMENT

SYNTAX OF IF-ELSE STATEMENT

```
if ( test expression 1)
{
    statement block 1;
}
else if ( test expression 2)
{
    statement block 2;
}
.....
else if (test expression N)
{
    statement block N;
}
else
{
    Statement Block X;
}
Statement Y;
```

3. IF ELSE IF STATEMENT

In the else if ladder construct, first the test expression is evaluated.

If the expression1 is true, statement block 1 is executed , if it is false then it will check expression2 and if it is true then statement block 2 is executed but if it is false then again it will check for next condition.

This process is continue up to last condition is checked. At last if all condition fails then it will be moved to else portion.

3. IF ELSE IF STATEMENT EXAMPLE

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a,b,c;
    clrscr();
    printf("\n Enter Value 1 : ");
    scanf("%d",&a);
    printf("\n Enter Value 2: ");
    scanf("%d",&b);
    printf("\n Enter Value 3: ");
    scanf("%d",&c);
    if(a>b && a>c)
    {
        printf("\n A is Maximum");
    }
    else if(b>a && b>c)
    {
        printf("\n B is Maximum");
    }
    else
    {
        printf("\n C is Maximum");
    }
}
```

4. NESTED IF ELSE

The if statement may itself contain another if statement is called as nested if..else statement.

Whenever we required to check condition inside other condition , such a situation is known as nested condition.

The general form of this statement is given as follows.

4. NESTED IF ELSE

SYNTAX OF NESTED IF-ELSE STATEMENT

```
if ( test expression 1)
{
    statement block 1;
    if(test expression 2)
    {
        statement block 2;
    }
    else
    {
        statement block 3;
    }
}
else
{
    Statement Block X;
}
Statement Y;
```

4. NESTED IF ELSE

In the nested if else statement, first the test expression1 is evaluated.

If the expression1 is true, statement block 1 is executed and it will check again for next test expression2 and if it is true then statement block 2 is executed but if it is false then statement block 3 is executed. But if Main test expression1 is false then it is directly move to the else part and execute remaining statement.

4. NESTED IF ELSE EXAMPLE

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a,b,c;
    clrscr();
    printf("\n Enter Value 1 : ");
    scanf("%d",&a);
    printf("\n Enter Value 2: ");
    scanf("%d",&b);
    printf("\n Enter Value 3: ");
    scanf("%d",&c);
    if(a>b)
    {
        if(a>c)
        {
            printf("\n A is Maximum");
        }
        else
        {
            printf("\n C is Maximum");
        }
    }
}
```

4. NESTED IF ELSE EXAMPLE

```
else
{
    if(b>c)
    {
        printf("\n B is Maximum");
    }
    else
    {
        printf("\n C is Maximum");
    }
}
getch();
}
```

SWITCH STATEMENT

Instead of using if-else-if ladder, the switch construct can be used to handle multiple choices, such as menu options. Using this statement we can develop a menu base program in C language.

The general syntax of the switch construct is different from if-else construct. The objective is to check several possible constant values for an expression.

SWITCH STATEMENT

```
switch (control variable )
{
    case constant-1 :
        statement block;
        break;
    case constant-2 :
        statement block;
        break;
    case constant-3 :
        statement block;
        break;
    .....
    .....

    default :
        statement block;
        [break];
}
```

SWITCH STATEMENT

The switch construct starts with the **switch** keyword followed by a different blocks which contain the different cases with constant value. Switch evaluates the control variable and compare with the first case if its value is equal to **constant-1**; then it executes the block of statements until it reaches the **break** keyword.

When the **break** keyword is found no more cases will be considered and the control is transferred out of the switch structure to next part of the program.

SWITCH STATEMENT

If the value of the expression is not equal to **constant-1** it will check the value of **constant-2**. If they are equal it will execute relevant statement(s). This process continuous until it finds a matching value. If a matching value is not found among any **cases**, the statement(s) given after the **default** keyword will be executed.

SWITCH STATEMENT

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a=0,b=0,ch=0,ans=0;
    clrscr();
    printf("Enetr A :");
    scanf("%d",&a);
    printf("Enetr B :");
    scanf("%d",&b);
    printf("\n-----\n");
    printf("Available choices");
    printf("\n-----\n");
    printf("\n1 - Addition (+)");
    printf("\n2 - Subtraction (-)");
    printf("\n3 - Multiplication (*)");
    printf("\n4 - Division (/)");
    printf("\n5 - Modules (%)");
    printf("\n\nEnetr your choice : ");
    scanf("%d",&ch);
```

SWITCH STATEMENT

```
switch(ch)
{
    case 1:
        ans=a+b;
        break;
    case 2:
        ans=a-b;
        break;
    case 3:
        ans=a*b;
        break;
    case 4:
        ans=a/b;
        break;
    case 5:
        ans=a%b;
        break;
}
printf("\nAnswer = %d",ans);
getch();
}
```

ITERATIVE STATEMENTS

Iterative statements are used to repeat the execution of a list of statements, depending on the value of an integer expression. In this section, we will discuss all these statements.

While loop

Do-while loop

For loop

ITERATIVE STATEMENTS

Actually there are two types of loop in C language Entry controlled loop and Exit controlled loop.

- **Entry controlled loop (pre test loop)** : In this loop, during the execution process of loop it first checks the condition and if the condition becomes true then and then it will execute body of the loop otherwise control will directly jump on next statement after the loop body. That's why this type of loop is known as Entry controlled loop.

For example : while loop and for loop

ITERATIVE STATEMENTS

- **Exit controlled loop** (Post test) : This type of loop first execute its body statements and at the end of body it check the condition. If condition becomes true then it will repeat the same process again or execute loop body once again. But if condition becomes false then it will terminate loop process and control will move on next statement. It means that this type of loop executes its statements at least one time and at the end of loop check the condition, that's why this type of loop is called exit controlled loop.

For Example : do... while loop

WHILE LOOP

- The while loop is used to repeat one or more statements while a particular condition is true.

```
Initialization of counter  
variable  
while (condition)  
{  
    statement_block;  
    increment/decrement;  
}  
statement x;
```

WHILE LOOP

- While loop is also entry controlled loop or pre test loop.
- In the while loop, the condition is tested before any of the statements in the statement block is executed.
- If the condition is true, only then the statements will be executed otherwise the control will jump to the immediate statement outside the while loop block.
- At last it will update counter variable by incrementing or decrementing.

WHILE LOOP

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i;
    i=1;
    while(i<=10)
    {
        printf("\n %d",i);
        i++;
    }
}
```

FOR LOOP

For loop is used to repeat a task until a particular condition is true.

The syntax of a for loop

```
for (initialization;condition;increment/decrement)  
{  
    statement block;  
}  
Statement Y;
```

FOR LOOP

For loop is also called entry controlled loop or pre test loop because it first check the condition and if condition becomes true then only execute its block.

The for loop allows us to specify three things about a loop in a single line

- (a) Setting a loop counter to an initial value.
- (b) Test Exp : Check the condition whether counter variable is reached at the desired value.
- (c) Increment or Decrement counter variable.

FOR LOOP

- In the for loop, first it is assigning value of variable then the condition is tested before any of the statements in the statement block is executed.
- If the condition is true, only then the statements will be executed otherwise the control will jump to the immediate statement outside the for loop block.
- At last it move to the increment/decrement portion and according to logic it will update the counter variable.

FOR LOOP

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i;

    for(i=1;i<=10;i++)
    {
        printf("\n %d",i);
    }
}
```

DO WHILE LOOP

The do-while loop is similar to the while loop. The only difference is that in a do-while loop, the test condition is tested at the end of the loop.

```
Statement x;  
Initialization of counter variable  
do  
{  
    statement_block;  
    increment/decrement  
} while (condition);  
statement y;
```

DO WHILE LOOP

Since the body of the loop is executed first and then the loop condition is checked in do while loop so it executes its statement at least once.

if condition becomes true then repeat same process again otherwise terminate the loop process. That's why do while loop is called an exit controlled loop.

DO WHILE LOOP

One important point is that , in case of for loop and while loop are not allowed ; (semicolon) at the end of loop statement but in case of do while loop it is required to write ; (semicolon) symbol at the end of condition.

The major disadvantage of using a do while loop is that it always executes at least once, so even if the user enters some invalid data, the loop will execute.

DO WHILE LOOP

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i;
    i=1;
    do
    {
        printf("\n %d",i);
        i++;
    }while(i<=10);
}
```

Break statement

The break statement is used to terminate the execution of the nearest enclosing loop in which it appears.

When compiler encounters a break statement, the control passes to the statement that follows the loop in which the break statement appears. Its syntax is quite simple, just type keyword break followed with a semi-colon.

```
break;
```

In switch statement if the break statement is missing then every case from the matched case label to the end of the switch, including the default, is executed.

Break statement

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i;

    for(i=1;i<=10;i++)
    {
        if(i==5)
            break;
        printf("\n %d",i);
    }
}
```

Continue statement

The continue statement can only appear in the body of a loop.

When the compiler encounters a continue statement then the rest of the statements in the loop are skipped and the control is unconditionally transferred to the loop-continuation portion of the nearest enclosing loop. Its syntax is quite simple, just type keyword continue followed with a semi-colon.

```
continue;
```

If placed within a for loop, the continue statement causes a branch to the code that updates the loop variable. For example,

Continue statement

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i;

    for(i=1;i<=10;i++)
    {
        if(i==5)
            continue;
        printf("\n %d",i);
    }
}
```

goto statement

The **goto** statement is used to transfer control to a specified label.

Here label is an identifier that specifies the place where the branch is to be made. Label can be any valid variable name that is followed by a colon (:).

Note that label can be placed anywhere in the program either before or after the goto statement. Whenever the goto statement is encountered the control is immediately transferred to the statements following the label.

goto statement

goto statement breaks the normal sequential execution of the program.

If the label is placed after the goto statement then it is called a forward jump and in case it is located before the goto statement, it is said to be a backward jump.

goto statement

```
#include <stdio.h>
#include <conio.h>

void main()
{
    int num, sum=0;
    read: // label for go to statement
    printf("\n Enter the number. Enter 999 to end : ");
    scanf("%d", &num);

    if (num != 999)
    {
        sum = sum+num;
        goto read; // jump to label- read
    }
    printf("\n Sum of the numbers entered by the user is = %d",
sum);
}
```

Thank You